

CRUD DE PERSONAS CON SPRING



Lección: Modificar Persona con Spring Boot y Thymeleaf

Introducción

En esta sección completaremos el segundo caso de uso del CRUD: **modificar una persona existente**. Aprovecharemos la misma vista que usamos para crear, lo que simplifica el mantenimiento y mejora la cohesión de la aplicación.

Paso 1: Agregar el botón "Editar" en la vista principal

 Abrimos `src/main/resources/templates/index.html`.

Agregamos una nueva columna en la tabla para editar una persona ya existente:

```
<th>Editar</th>
```

Dentro del bucle que recorre la lista de personas, agregamos un enlace para editar:

```
<td>
  <a th:href="@{'/editar/' + ${persona.idPersona}}">Editar</a>
</td>
```

Esto genera una URL como `/editar/1` que será procesada por el controlador para cargar la persona correspondiente.

⚙ Paso 2: Crear el método GET `/editar/{idPersona}` en el controlador

📁 Abrimos `src/main/java/gm/web/ControladorInicio.java`.

Agregamos este método:

```
@GetMapping("/editar/{idPersona}")
public String editar(Persona persona, Model model) {
    persona = personaService.encontrarPersona(persona);
    model.addAttribute("persona", persona);
    return "modificar";
}
```

🔍 Spring inyecta automáticamente el `idPersona` recibido desde la URL dentro del objeto `persona`. Con eso, se hace la búsqueda en la base de datos y se comparte la persona completa al modelo.

🔧 Paso 3: Mostrar los valores en el formulario `modificar.html`

📁 Abrimos `src/main/resources/templates/modificar.html`.

Ya teníamos configurado el formulario, pero ahora debemos asegurarnos de que se envíe el valor de `idPersona`:

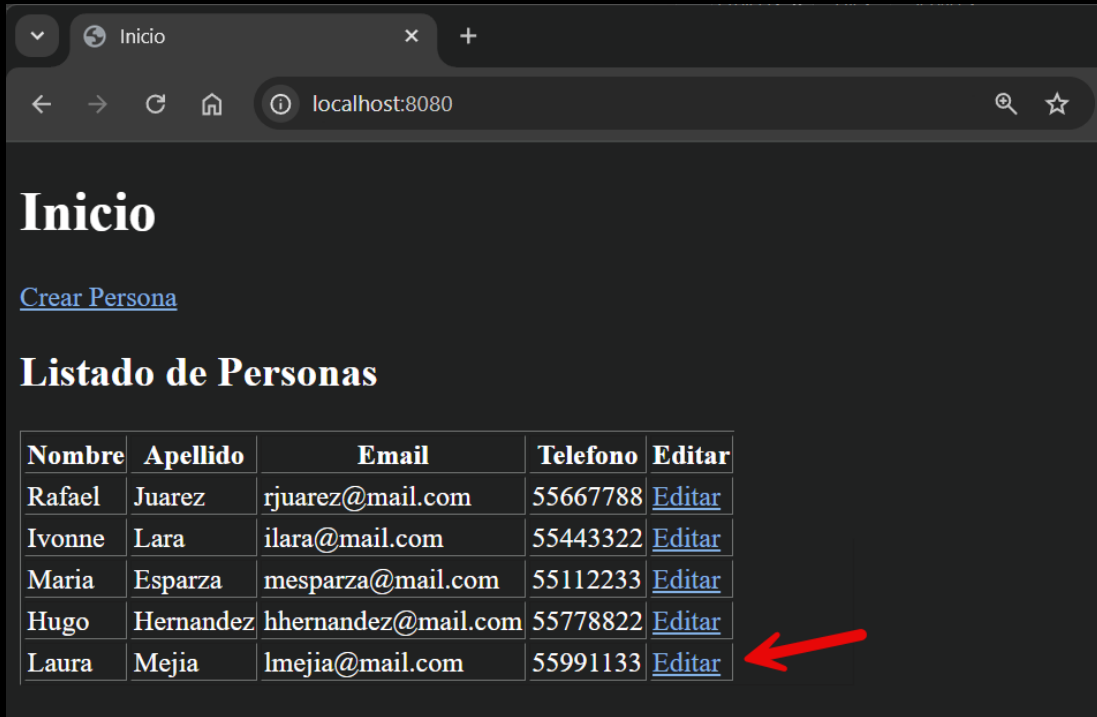
```
<input type="hidden" th:field="*{idPersona}" />
```

Este campo oculto es **clave** para que Spring sepa si debe hacer un `INSERT` o un `UPDATE`.

Paso 4: Probar modificación

1. Inicia la aplicación.
2. Ve a la página de inicio: `http://localhost:8080/`.
3. Haz clic en “Editar” sobre un registro existente.
4. Cambia uno o más datos y presiona **Guardar**.

✓ Si todo está bien, los cambios se verán reflejados en el listado.



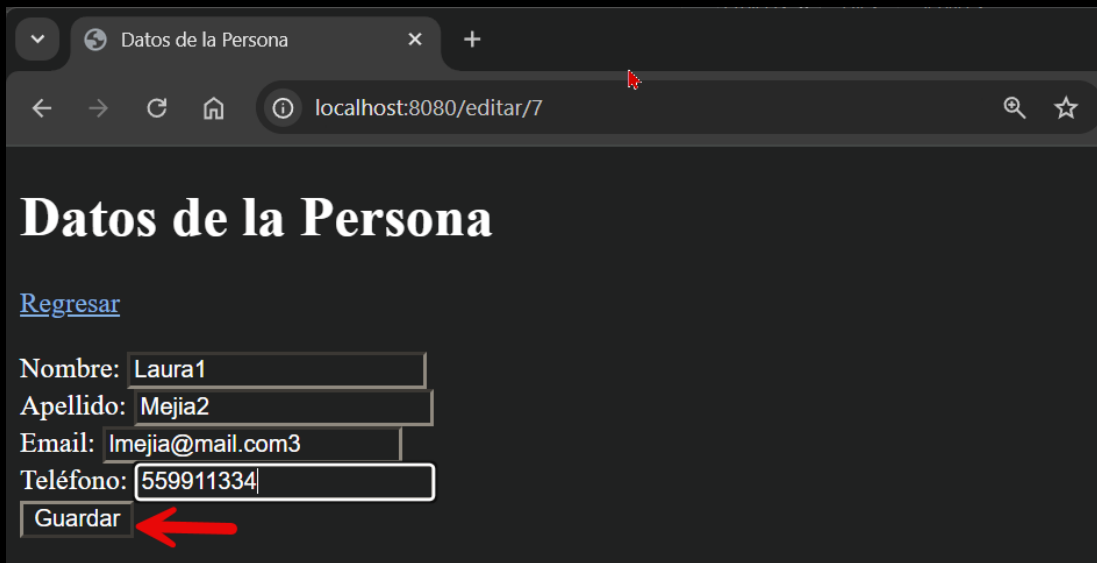
Inicio

[Crear Persona](#)

Listado de Personas

Nombre	Apellido	Email	Telefono	Editar
Rafael	Juarez	rjuarez@mail.com	55667788	Editar
Ivonne	Lara	ilara@mail.com	55443322	Editar
Maria	Esparza	mesparza@mail.com	55112233	Editar
Hugo	Hernandez	hhernandez@mail.com	55778822	Editar
Laura	Mejia	lmejia@mail.com	55991133	Editar

Realizamos cualquier cambio sólo para comprobar que se guarden correctamente los cambios en la base de datos y se vean las sentencias SQL en el log de la aplicación.



Datos de la Persona

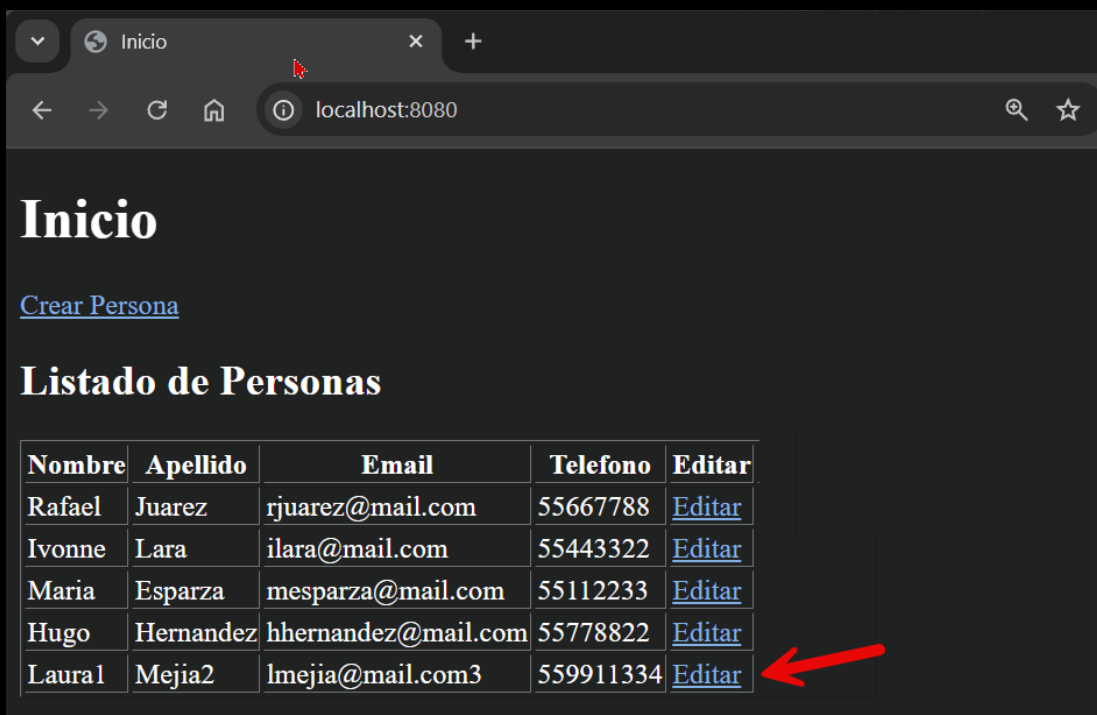
[Regresar](#)

Nombre:

Apellido:

Email:

Teléfono:



Inicio

[Crear Persona](#)

Listado de Personas

Nombre	Apellido	Email	Telefono	Editar
Rafael	Juarez	rjuarez@mail.com	55667788	Editar
Ivonne	Lara	ilara@mail.com	55443322	Editar
Maria	Esparza	mesparza@mail.com	55112233	Editar
Hugo	Hernandez	hhernandez@mail.com	55778822	Editar
Laura1	Mejia2	lmejia@mail.com3	559911334	Editar

Si lo deseamos, regresamos los valores originales de registro editando nuevamente la persona modificada, una vez confirmado que todo está funcionando correctamente.

Conclusión

Con estos pasos, implementamos correctamente el caso de **modificar personas** usando una misma vista para crear y editar, diferenciando los casos con un campo oculto para `idPersona`. Esto es posible gracias a la magia de Thymeleaf + Spring Boot, que simplifica la gestión de formularios y objetos.

✚ Código completo `index.html`

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Inicio</title>
    <meta charset="UTF-8">
  </head>
  <body>
    <h1>Inicio</h1>
    <a th:href="@{/agregar}">Crear Persona</a>
    <h2>Listado de Personas</h2>
    <div th:if="{personas != null and !personas.empty}">
      <table border="1">
        <tr>
          <th>Nombre</th>
          <th>Apellido</th>
          <th>Email</th>
          <th>Telefono</th>
          <th>Editar</th>
        </tr>
        <tr th:each="persona : {personas}">
          <td th:text="{persona.nombre}">Nombre</td>
          <td th:text="{persona.apellido}">Apellido</td>
          <td th:text="{persona.email}">Email</td>
          <td th:text="{persona.telefono}">Telefono</td>
          <td><a th:href="@{/editar/} + {persona.idPersona}" th:text="Editar"/></td>
        </tr>
      </table>
    </div>
    <div th:if="{personas == null or personas.empty}">
      La lista de personas está vacía
    </div>
  </body>
</html>
```

✚ Código completo `ControladorInicio.html`

```
package gm.web;

import gm.domain.Persona;
import gm.servicio.PersonaService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class ControladorInicio {

    private static final Logger log = LoggerFactory.getLogger(ControladorInicio.class);

    @Autowired
    private PersonaService personaService;

    @GetMapping("/")
    public String inicio(Model model) {
        log.info("Ejecutando el controlador Spring MVC");
        var personas = personaService.listarPersonas();
        model.addAttribute("personas", personas);
        return "index";
    }

    @GetMapping("/agregar")
    public String agregar(Persona persona) {
        return "modificar";
    }

    @PostMapping("/guardar")
    public String guardar(Persona persona) {
        personaService.guardar(persona);
        return "redirect:/";
    }

    @GetMapping("/editar/{idPersona}")
    public String editar(Persona persona, Model model) {
        persona = personaService.encontrarPersona(persona);
        model.addAttribute("persona", persona);
        return "modificar";
    }
}
```

}

🔗 Código completo modificar.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org"
      xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>Datos de la Persona</title>
    <meta charset="UTF-8"/>
  </head>
  <body>
    <h1>Datos de la Persona</h1>
    <a th:href="@{/}">Regresar</a>
    <br/>
    <form th:action="@{/guardar}" method="post" th:object="${persona}">
      <input type="hidden" name="idPersona" th:field="*{idPersona}"/>
      <br/>
      <label for="nombre">Nombre:</label>
      <input type="text" name="nombre" th:field="*{nombre}"/>
      <br/>
      <label for="apellido">Apellido:</label>
      <input type="text" name="apellido" th:field="*{apellido}"/>
      <br/>
      <label for="email">Email:</label>
      <input type="email" name="email" th:field="*{email}"/>
      <br/>
      <label for="telefono">Teléfono:</label>
      <input type="tel" name="telefono" th:field="*{telefono}"/>
      <br/>
      <input type="submit" name="guardar" value="Guardar"/>
    </form>
  </body>
</html>
```

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)